Implementación de dbt-expectations

Actividad Asincrónica - dbt Testing

1. Configuración de Tests Genéricos y dbt-expectations

Se implementaron validaciones de integridad técnica y descriptiva en las capas de Staging y Marts. Para el campo `tc_promedio`, se utilizó una severidad de advertencia (`warn`) para permitir la continuidad del proceso ante valores atípicos que se generaron de manera intecional para validar la funcionalidad de las validaciones.

```yaml
version: 2

models:
  - name: stg_tipo_cambio
    description: "Limpieza y desagruacion del JSON de tipo de cambio"
    tests:
      # dbt-expectations: Valida que la tabla no esté vacía
      - dbt_expectations.expect_table_row_count_to_be_between:
          min_value: 1
          severity: warn
    columns:
      - name: fecha_cotizacion
        tests:
          - not_null
      - name: moneda_destino
        tests:
          - unique
          - not_null
      - name: tipo_cambio
        tests:
          - not_null
          # dbt-expectations: El tipo de cambio debe ser mayor a cero
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
```

Figure 1: Tests en Capa Staging

```yaml
version: 2

models:
  - name: obt_tipo_cambio
    description: "Tabla final consolidada"
    columns:
      - name: moneda_base
        tests:
          # Solo permitir  estas monedas
          - accepted_values:
              values: ['USD', 'EUR', 'PYG']
      - name: tc_promedio
        tests:
          # dbt-expectations: El promedio debe estar en un rango lógico
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
              max_value: 100000
              severity: warn #avisar pero no detener el proceso
```

Figure 2: Tests en Capa Marts

2. Singular Tests (Validaciones de Negocio)

Se crearon dos tests personalizados para validar reglas de negocio críticas: asegurar que no existan tasas negativas y que la moneda base sea distinta a la de destino. Ambos fueron configurados con `severity: warn`.

```sql
-- Regla de negocio: Validar que el tipo de cambio siempre debe ser un valor positivo.
-- El test fallará si encuentra algún registro con tasa <= 0.
{{ config(severity = 'warn') }}
select
    moneda_destino,
    tipo_cambio
from {{ ref('stg_tipo_cambio') }}
where tipo_cambio <= 0
```

Figure 3: Test: Tipo de cambio Positivo

```sql
-- Regla de negocio: Validar que la moneda de origen no sea igual a la de destino.
-- El test fallará si encuentra filas donde coincidan.
{{ config(severity = 'warn') }}
select
    moneda_base,
    moneda_destino,
    fecha_cotizacion
from {{ ref('stg_tipo_cambio') }}
where moneda_base = moneda_destino
```

Figure 4: Test: Monedas Distintas

3. Ejecución y Resultados (`dbt build`)

La ejecución mediante `dbt build` demuestra que el pipeline es capaz de procesar 15 nodos (seeds, modelos y tests). Se observan 2 advertencias gestionadas correctamente, lo que garantiza la disponibilidad de los datos finales.

```
21:50:01  1 of 15 START sql view model main.stg_tipo_cambio ............................ [RUN]
21:50:03  1 of 15 OK created sql view model main.stg_tipo_cambio ....................... [OK in 1.46s]
21:50:03  2 of 15 START seed file main.ref_monedas ..................................... [RUN]
21:50:04  2 of 15 OK loaded seed file main.ref_monedas ................................ [INSERT 61 in 0.99s]
21:50:04  3 of 15 START test assert_monedas_distintas .................................... [RUN]
21:50:04  3 of 15 WARN 1 assert_monedas_distintas ..................................... [WARN 1 in 0.26s]
21:50:04  4 of 15 START test assert_tipo_cambio_positivo ................................ [RUN]
21:50:04  4 of 15 PASS assert_tipo_cambio_positivo ..................................... [PASS in 0.33s]
21:50:04  5 of 15 START test dbt_expectations_expect_column_values_to_be_between_stg_tipo_cambio_tipo_cambio__0  [RUN]
21:50:05  5 of 15 PASS dbt_expectations_expect_column_values_to_be_between_stg_tipo_cambio_tipo_cambio__0  [PASS in 0.24s]
21:50:05  6 of 15 START test dbt_expectations_expect_table_row_count_to_be_between_stg_tipo_cambio_1  [RUN]
21:50:05  6 of 15 PASS dbt_expectations_expect_table_row_count_to_be_between_stg_tipo_cambio_1  [PASS in 0.22s]
21:50:05  7 of 15 START test not_null_stg_tipo_cambio_fecha_cotizacion ................... [RUN]
21:50:05  7 of 15 PASS not_null_stg_tipo_cambio_fecha_cotizacion ....................... [PASS in 0.21s]
21:50:05  8 of 15 START test not_null_stg_tipo_cambio_moneda_destino ..................... [RUN]
21:50:05  8 of 15 PASS not_null_stg_tipo_cambio_moneda_destino ........................ [PASS in 0.23s]
21:50:05  9 of 15 START test not_null_stg_tipo_cambio_tipo_cambio ...................... [RUN]
21:50:05  9 of 15 PASS not_null_stg_tipo_cambio_tipo_cambio ......................... [PASS in 0.21s]
21:50:05  10 of 15 START test unique_stg_tipo_cambio_moneda_destino ..................... [RUN]
21:50:06  10 of 15 PASS unique_stg_tipo_cambio_moneda_destino ........................ [PASS in 0.21s]
21:50:06  11 of 15 START sql view model main.stg_monedas ............................... [RUN]
21:50:07  11 of 15 OK created sql view model main.stg_monedas ......................... [OK in 1.41s]
21:50:07  12 of 15 START sql view model main.int_tipo_cambio_mensual ..................... [RUN]
21:50:08  12 of 15 OK created sql view model main.int_tipo_cambio_mensual ............... [OK in 1.33s]
21:50:08  13 of 15 START sql view model main.obt_tipo_cambio ............................ [RUN]
21:50:10  13 of 15 OK created sql view model main.obt_tipo_cambio ....................... [OK in 1.33s]
21:50:10  14 of 15 START test accepted_values_obt_tipo_cambio_moneda_base__USD__EUR__PYG . [RUN]
21:50:10  14 of 15 PASS accepted_values_obt_tipo_cambio_moneda_base__USD__EUR__PYG ..... [PASS in 0.22s]
21:50:10  15 of 15 START test dbt_expectations_expect_column_values_to_be_between_obt_tipo_cambio_tc_promedio__100000__0  [RUN]
21:50:10  15 of 15 WARN 2 dbt_expectations_expect_column_values_to_be_between_obt_tipo_cambio_tc_promedio__100000__0  [WARN 2 in 0.23s]
21:50:10
21:50:10  Finished running 1 seed, 10 data tests, 4 view models in 0 hours 0 minutes and 12.12 seconds (12.12s).
21:50:10
21:50:10  Completed with 2 warnings:
21:50:10
21:50:10  Warning in test assert_monedas_distintas (tests\assert_monedas_distintas.sql)
21:50:10  Got 1 result, configured to warn if != 0
21:50:10
21:50:10    compiled code at target\compiled\mi_proyecto_dbt\tests\assert_monedas_distintas.sql
21:50:10
21:50:10  Warning in test dbt_expectations_expect_column_values_to_be_between_obt_tipo_cambio_tc_promedio__100000__0 (models\marts\marts_exchange_rates.yml)
21:50:10  Got 2 results, configured to warn if != 0
21:50:10
21:50:10    compiled code at target\compiled\mi_proyecto_dbt\models\marts\marts_exchange_rates.yml\dbt_expectations_expect_column_c46cc7cc50861c5a61ccd2ad61fefcc3.sql
21:50:10
21:50:10  Done. PASS=13 WARN=2 ERROR=0 SKIP=0 NO-OP=0 TOTAL=15
```

Figure 5: Resultado de dbt build con 13 PASS y 2 WARN.

4. Linaje de Datos con Nodos de Calidad

El grafo de dependencias (DAG) generado muestra la integración de los nuevos tests dentro del flujo de transformación, permitiendo una trazabilidad completa.

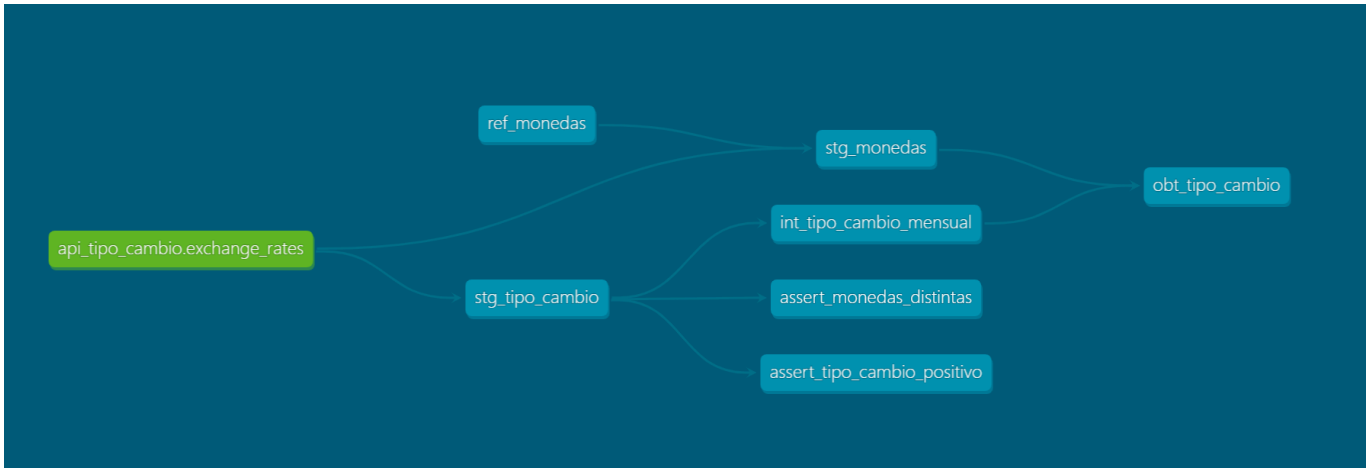

Figure 6: DAG interactivo incluyendo validaciones personalizadas.

5. Lógica de Transformación (Capa Intermediate)

SQL utilizada para las agregaciones mensuales, asegurando que el cálculo de promedios, máximos y mínimos sea consistente antes de llegar a la capa Mart.

```sql
with tipo_cambio as (
    select * from {{ ref('stg_tipo_cambio') }}
),

calculo as (
    select
        moneda_base,
        moneda_destino,
        date_trunc('month', fecha_cotizacion) as mes_cotizacion,
        avg(tipo_cambio) as tc_promedio,
        max(tipo_cambio) as tc_maxima,
        min(tipo_cambio) as tc_minima,
        now() as ingresado_at
    from tipo_cambio
    group by 1, 2, 3
)

select * from calculo
```

Figure 7: SQL del modelo intermediate de agregación mensual.